

TRABAJO PRÁCTICO INTEGRADOR

Programación 1

Sistema de Gestión de Datos de Países (TP Integrador)

Alumnos: Fabricio Emanuel Diaz Cornejo
Tomas Agostino D'Inca Ponte

Institución: **Universidad Tecnológica Nacional - Tecnicatura Universitaria en Programación**

Fecha de entrega: 17 de Junio de 2026

Índice

- 1. Introducción: 3**
- 2. Marco Teórico: 3-5**
- 3. Descripción del Sistema y flujo: 6-7**
- 4. Decisiones Técnicas: 8**
- 5. Dificultades Tecnicas: 9**
- 6. Conclusiones: 10**
- 7. Referencias: 11**
- 8. Links: 11**

1. Introducción

El presente informe documenta el desarrollo del Trabajo Práctico Integrador de la materia Programación 1. El proyecto consiste en un sistema de gestión de datos de países implementado en el lenguaje Python, que opera completamente por línea de comandos (consola).

El sistema permite al usuario realizar operaciones de alta, consulta, modificación, filtrado y ordenamiento sobre un conjunto de datos de países reales almacenados en un archivo con formato CSV. Además, calcula estadísticas básicas como promedios y valores extremos.

El objetivo principal del trabajo es aplicar de manera integrada los conceptos fundamentales del paradigma imperativo/estructurado estudiados a lo largo del cuatrimestre.

2. Marco Teórico

A continuación se presentan y definen los conceptos fundamentales de Python utilizados en el desarrollo del sistema.

2.1 Listas

En Python, una lista es una estructura de datos ordenada y mutable que puede contener elementos de cualquier tipo. Se define utilizando corchetes (`[]`) y los elementos se separan por comas.

Las listas son mutables, lo que significa que sus elementos pueden ser modificados, agregados o eliminados después de su creación. En este proyecto, se utiliza una lista como contenedor principal de todos los países cargados en memoria, permitiendo iterar sobre ella, filtrarla y ordenarla con facilidad.

Ejemplo de lista utilizada en el proyecto:

```
países = [] # Lista vacía que se rellena al leer el CSV
países.append(nuevo_país) # Agrega un nuevo país a la lista
```

2.2 Diccionarios

Un diccionario en Python es una estructura de datos que almacena pares clave-valor. Se define con llaves (`{ }`) y cada elemento tiene una clave única que permite acceder a su valor asociado.

Son estructuras ideales para representar registros o entidades con múltiples atributos. En este trabajo, cada país se representa como un diccionario con cuatro claves: nombre, población, superficie y continente. Esta estructura facilita el acceso a cada campo por su nombre descriptivo, haciendo el código más legible y mantenible.

Ejemplo:

```
país = { "nombre": "Argentina", "poblacion": 45195774, "superficie": 2780400, "continente": "América" }
```

2.3 Funciones y Modularización

Una función es un bloque de código reutilizable que realiza una tarea específica. En Python se define con la palabra clave `def`, seguida del nombre de la función y sus parámetros entre paréntesis.

El diseño del sistema sigue el principio de responsabilidad única: cada función realiza una sola tarea claramente definida. Esto facilita la lectura del código, su mantenimiento y la detección de errores. El proyecto contiene más de 15 funciones organizadas en módulos lógicos, tales como carga de datos, validación de entradas, búsqueda, filtros, ordenamiento y estadísticas.

Ejemplo:

```
def cargar_csv(nombre_archivo):  
    """Lee el archivo CSV y retorna una lista de diccionarios."""  
    ...
```

2.4 Estructuras Condicionales

Las estructuras condicionales permiten que el programa tome decisiones en función de si una condición es verdadera o falsa. Python utiliza las palabras reservadas `if`, `elif` y `else` para este propósito.

En este proyecto, los condicionales se usan ampliamente para validar entradas del usuario, verificar si un país ya existe antes de agregarlo, y dirigir el flujo del programa según la opción seleccionada en el menú.

Ejemplo:

```
if encontrar_pais_exacto(paises, nombre):  
    print("El país ya existe.")  
else:  
    paises.append(nuevo_pais)
```

2.5 Bucles (while y for)

Los bucles permiten repetir un bloque de código mientras se cumpla una condición (`while`) o para cada elemento de una colección (`for`).

El bucle `while` principal del programa mantiene el menú activo hasta que el usuario elige la opción de salir. Los bucles `for` se utilizan para iterar sobre la lista de países al momento de filtrar, imprimir o calcular estadísticas.

Ejemplo del bucle principal:

```
while True:  
    mostrar_menu_principal()  
    opcion = pedir_opcion_menu(opciones_validas)  
    if opcion == "0":  
        break    # Sale del bucle y termina el programa
```

2.6 Ordenamiento con Funciones Lambda

Python provee la función incorporada `sorted()`, que devuelve una nueva lista ordenada a partir de cualquier iterable. La clave de ordenamiento se define mediante el parámetro `key`, al que se le puede pasar una función lambda.

Una función lambda es una función anónima de una sola expresión. En este proyecto, se utilizan para indicar por qué campo del diccionario se desea ordenar, y el parámetro `reverse` permite elegir entre orden ascendente o descendente.

Ejemplo:

```
lista_ordenada = sorted(paises, key=lambda p: p["poblacion"], reverse=True)
```

2.7 Estadísticas Básicas

El módulo de estadísticas calcula métricas fundamentales sobre el conjunto de datos sin utilizar librerías externas, aplicando las funciones nativas de Python:

- `max()` y `min()` con clave lambda para encontrar el país con mayor y menor población.
- `sum()` con una expresión generadora para sumar las poblaciones o superficies.
- División entera (`//`) para calcular promedios.
- Un diccionario de conteo para acumular la cantidad de países por continente.

2.8 Manejo de Archivos CSV

CSV es un formato de texto plano para almacenar datos tabulares, donde cada línea representa una fila y los valores se separan por comas. Es ampliamente utilizado por su simplicidad y compatibilidad.

Python provee el módulo `csv` en su biblioteca estándar, que facilita la lectura y escritura de estos archivos. Se utilizó `csv.DictReader` para cargar los datos como diccionarios directamente y `csv.DictWriter` para guardar los cambios respetando el orden de las columnas.

Ejemplo de lectura:

```
with open("paises.csv", mode="r", encoding="utf-8") as archivo:
    lector = csv.DictReader(archivo)
    for fila in lector:
        paises.append({ "nombre": fila["nombre"], ... })
```

2.9 Manejo de Excepciones (try-except)

Los bloques `try-except` son el mecanismo de Python para gestionar errores en tiempo de ejecución. El código que podría fallar se coloca dentro del bloque `try`; si ocurre un error, el programa pasa al bloque `except` en lugar de interrumpirse abruptamente.

En este sistema, se aplica manejo de excepciones en múltiples puntos críticos: la conversión de texto a entero (que puede fallar si el usuario escribe letras), la apertura del archivo CSV (que puede no existir) y la escritura del archivo (que puede fallar por permisos).

Ejemplo:

```
try:
    valor = int(input('Ingrese un número: '))
except ValueError:
    print('Error: debe ingresar un número entero.')
```

3. Descripción del Sistema

3.1 Arquitectura General

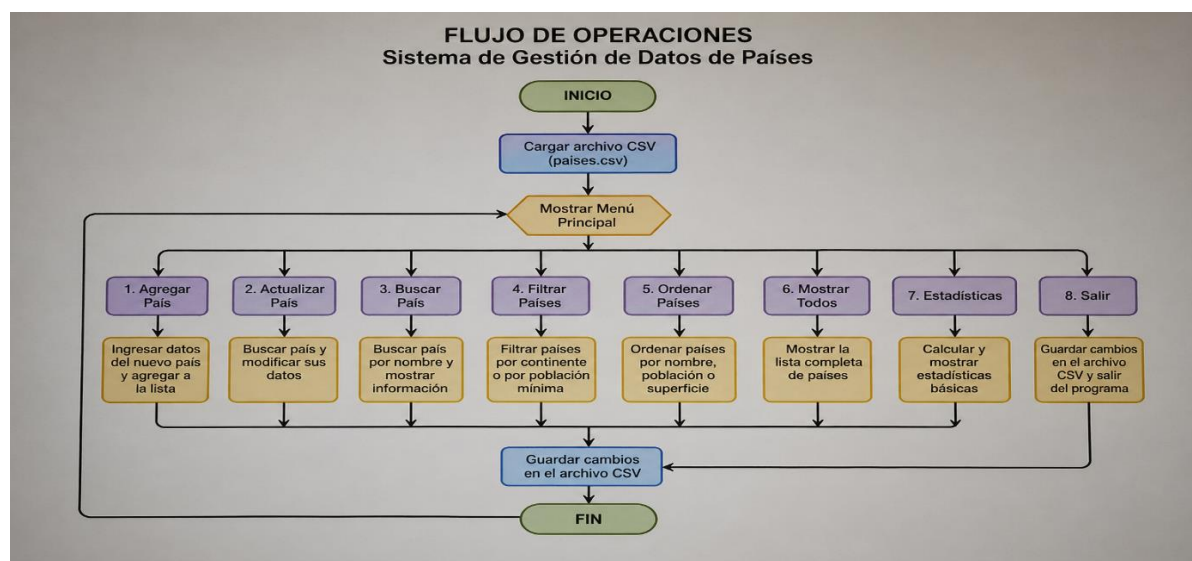
El programa está organizado en 10 módulos funcionales claramente diferenciados dentro de un único archivo main.py, siguiendo el principio de separación de responsabilidades:

Módulo	Responsabilidad
Módulo 1: Entrada y Salida CSV	cargar_csv() y guardar_csv()
Módulo 2: Visualización	Funciones de impresión con formato
Módulo 3: Validaciones	pedir_texto(), pedir_entero_positivo()
Módulo 4: Búsqueda	buscar_pais_por_nombre(), encontrar_pais_exacto()
Módulo 5: Filtros	Filtros por continente, población y superficie
Módulo 6: Ordenamiento	ordenar_paises() con lambda
Módulo 7: Estadísticas	calcular_estadisticas()
Módulos 8-10: Alta, Baja, Modificaicon y Menú	Lógica de alta, modificación y menú principal

3.2 Flujo del Programa

Al ejecutarse, el programa sigue este flujo:

- 1. main() invoca cargar_csv(), que lee el archivo paises.csv y llena la lista de diccionarios paises en memoria.
- 2. Se lanza el bucle while principal en ejecutar_menu_principal(), que se mantiene activo hasta que el usuario elige la opción 0 (Salir).
- 3. En cada iteración, se muestra el menú y se valida la opción ingresada.
- 4. La función correspondiente a la opción elegida es invocada, recibiendo la lista paises como argumento.
- 5. Si se agrega o modifica un país, guardar_csv() persiste los cambios en el archivo.



4. Decisiones Técnicas

4.1 Lista de Diccionarios como Estructura de Datos Principal

La decisión de utilizar una lista de diccionarios para almacenar los países en memoria fue tomada por diversas razones que la convierten en la opción más adecuada para este nivel de programación.

En primer lugar, cada país es una entidad con múltiples atributos (nombre, población, superficie, continente). El diccionario es la estructura natural en Python para representar este tipo de registros, ya que permite acceder a cada campo por su nombre (`pais["nombre"]`) en lugar de por un índice numérico (`pais[0]`), lo que hace el código mucho más legible y menos propenso a errores.

En segundo lugar, al agrupar estos diccionarios en una lista, se obtiene una colección ordenada y mutable de todos los países, sobre la que se pueden aplicar fácilmente las operaciones de iteración (`for`), filtrado (`list comprehensions`), ordenamiento (`sorted`) y búsqueda que el sistema requiere.

Finalmente, esta combinación (lista de diccionarios) es la forma más directa de trabajar con los datos que provienen de un archivo CSV leído con `csv.DictReader`, ya que ese módulo devuelve cada fila como un diccionario, lo que simplifica notablemente el código de carga de datos.

4.2 Guardado Automático ante Cambios

Se implementó un mecanismo de guardado automático: cada vez que el usuario agrega un nuevo país o actualiza los datos de uno existente, la función `guardar_csv()` es llamada automáticamente. Esto garantiza que los cambios no se pierdan si el programa se cierra de manera inesperada, ofreciendo una experiencia más robusta y consistente.

5. Dificultades Encontradas

5.1 Conversión de Tipos al Leer el CSV

Una de las primeras dificultades encontradas fue que el módulo csv de Python lee todos los valores del archivo como cadenas de texto (strings), independientemente de su contenido. Por lo tanto, los campos población y superficie llegaban como "45195774" (string) en lugar de 45195774 (entero).

Esto significaba que operaciones como comparaciones numéricas para filtrar por rango o el cálculo de promedios fallaban de maneras inesperadas. La solución fue aplicar la conversión `int()` explícitamente al momento de cargar cada fila, dentro de un bloque `try-except` para capturar posibles errores si el archivo CSV tuviera datos corruptos o con formato incorrecto.

5.2 Validación de Entradas del Usuario

Otro desafío significativo fue diseñar un sistema de validación de entradas robusto. El problema central es que `input()` en Python siempre devuelve un string, y si el usuario escribe "abc" cuando se espera un número, la conversión `int()` lanza una excepción `ValueError` que, sin manejo, interrumpe el programa.

La solución fue encapsular toda la lógica de solicitud de datos en funciones de validación especializadas (`pedir_texto()`, `pedir_entero_positivo()`) que utilizan bucles `while True` con bloques `try-except` internos, garantizando que el programa solo avanza cuando recibe un dato válido y mostrando mensajes de error claros al usuario.

5.3 Modularización del Código

Al comenzar el desarrollo, existía la tentación de escribir el código de manera lineal, colocando toda la lógica en pocas funciones grandes. Sin embargo, esto generaba funciones difíciles de leer y de depurar.

La reestructuración del código en pequeñas funciones con responsabilidades únicas requirió un esfuerzo de planificación adicional (decidir qué debe hacer cada función, qué parámetros recibe y qué devuelve), pero el resultado final es un código mucho más claro, mantenible y fácil de extender con nuevas funcionalidades.

6. Conclusiones

El desarrollo de este Trabajo Práctico Integrador permitió consolidar los conocimientos adquiridos a lo largo de la materia Programación 1, integrando de manera práctica y concreta los conceptos de variables, tipos de datos, estructuras condicionales, bucles, funciones, listas, diccionarios, manejo de archivos y excepciones.

El proyecto demostró que Python es un lenguaje altamente expresivo y que sus estructuras de datos nativas (especialmente las listas y los diccionarios) son herramientas poderosas y versátiles que permiten resolver problemas reales de gestión de información sin necesidad de recurrir a librerías externas.

La aplicación del principio de modularización fue uno de los factores más importantes: dividir un problema complejo en funciones pequeñas y bien definidas no solo facilita el desarrollo, sino que también simplifica enormemente la detección y corrección de errores.

Finalmente, el desafío de las validaciones de entrada y el manejo de excepciones enseñó una lección fundamental de la programación real: un programa no solo debe funcionar correctamente cuando el usuario hace lo esperado, sino que también debe comportarse de manera segura y predecible cuando recibe datos incorrectos o inesperados.

7. Referencias

- Material de Catedra Programación 1, Universidad Tecnológica Nacional
- [Ficheros csv \(escritura\) – Python 3 para todo](#)
- [Python Lambda](#)

8. Links

- Video: <https://youtu.be/-fRVQhQROjc>
- GitHub: <https://github.com/FabricioDiazC/Trabajo-Integrador-Programacion-1-2026.git>